

گزارش تمرین ۲

مهسا ابرغانی اقدام – شماره دانشجویی : ۸۱۰۱۰۳۰۵۳

الف:

توابع کمکی:

- `train_test_split_manual`: این تابع داده‌های ورودی را به دو بخش آموزشی و آزمایشی به نسبت مشخص تقسیم می‌کند.
- `compute_gradient`: این تابع گرادیان تابع هزینه را برای به‌روزرسانی وزن‌ها محاسبه می‌کند.
- `update_weights`: در این تابع وزن‌های مدل با استفاده از گرادیان و نرخ یادگیری (`lr`) به‌روزرسانی می‌شوند.
- `evaluate_model`: این تابع دقت مدل را با محاسبه خروجی‌های پیش‌بینی شده و مقایسه آن‌ها با مقادیر واقعی محاسبه می‌کند.

رگرسیون لجستیک توزیع‌شده:

- تابع `distributed_logistic_regression`: این تابع پیاده‌سازی رگرسیون لجستیک را به‌صورت توزیع‌شده و موازی انجام می‌دهد. در اینجا از `MPI` برای تقسیم داده‌ها بین پروسس‌ها و جمع‌آوری گرادیان‌ها استفاده شده است.
 - هر پردازش بخشی از داده‌ها را دریافت کرده و روی آن‌ها عملیات انجام می‌دهد.
 - هر پروسس گرادیان خود را محاسبه می‌کند و سپس گرادیان‌ها با استفاده از تابع `Allreduce` در `MPI` جمع می‌شوند تا وزن‌ها به‌روزرسانی شوند.

پارامترها:

- نرخ یادگیری (`lr`) با مقدار پیش‌فرض ۰.۰۱ و تعداد تکرارها (`epochs`) با مقدار ۱۰۰ به‌عنوان پارامترهای کلیدی الگوریتم در نظر گرفته شده‌اند.
- از داده‌های ذخیره شده در فایل‌های `data.npy` و `labels.npy` برای آموزش و آزمایش مدل استفاده می‌شود.

قبل و بعد از فراخوانی داده‌ها و اجرای طبقه‌بندی، زمان سیستم را برای بدست آوردن زمان اجرا ذخیره می‌کنیم. در نهایت با استفاده از تابع print نتیجه تابع taylor و زمان اجرا چاپ می‌شود.

```
2 > الف > LogReg_a.py > [x] startTime
1  import time
2  from mpi4py import MPI
3  import numpy as np
4  import os
5
6
7  def train_test_split_manual(X, Y, test_size=0.2):
8      data_size = X.shape[0]
9      test_size = int(data_size * test (variable) data_size: Any
10     indices = np.random.permutation(data_size)
11     i_test = indices[:test_size]
12     i_train = indices[test_size:]
13     X_train, X_test = X[i_train], X[i_test]
14     Y_train, Y_test = Y[i_train], Y[i_test]
15     return X_train, X_test, Y_train, Y_test
16
17  def compute_gradient(X, Y, weights):
18     predicted = 1 / (1 + np.exp(-np.dot(X, weights)))
19     errors = Y - predicted
20     gradient = -np.dot(X.T, errors) / len(Y)
21     return gradient
22
23  def update_weights(weights, gradient, lr):
24     weights -= lr * gradient
25     return weights
26
27  def evaluate_model(X, Y, weights):
28     predicted = 1 / (1 + np.exp(-np.dot(X, weights)))
29     output = (predicted >= 0.5).astype(int)
30     accuracy = np.mean(output == Y)
```

```

2 > الف > LogReg_a.py > startTime
29     output = (predicted >= 0.5).astype(int)
30     accuracy = np.mean(output == Y)
31     return accuracy
32
33     def distributed_logistic_regression(X, Y, Lr, epochs):
34         comm = MPI.COMM_WORLD
35         rank = comm.Get_rank()
36         size = comm.Get_size()
37
38         total_features = X.shape[1]
39         weights = np.zeros(total_features)
40
41         data_size = X.shape[0]
42         sub_size = data_size // size
43
44         local_X = X[rank * sub_size:(rank + 1) * sub_size]
45         local_Y = Y[rank * sub_size:(rank + 1) * sub_size]
46
47         for epoch in range(epochs):
48             sub_gradient = compute_gradient(local_X, local_Y, weights)
49             total_gradient = np.zeros_like(sub_gradient)
50             comm.Allreduce(sub_gradient, total_gradient, op=MPI.SUM)
51             weights = update_weights(weights, total_gradient / size, Lr)
52
53         return weights
54
55     lr = 0.01
56     epochs = 100
57
58     startTime = time.time()

```

```

2 > الف > LogReg_a.py > [?] startTime
57
58 startTime = time.time()
59
60 data = np.load('/home/dmls/aqdam/tmrine1/2/a/data.npy')
61 labels = np.load('/home/dmls/aqdam/tmrine1/2/a/labels.npy')
62
63 X_train, X_test, Y_train, Y_test = train_test_split_manual(data, labels)
64
65 comm = MPI.COMM_WORLD
66 rank = comm.Get_rank()
67
68 if rank == 0:
69     final_weights = distributed_logistic_regression(X_train, Y_train, lr, epochs)
70     test_accuracy = evaluate_model(X_test, Y_test, final_weights)
71     print("Accuracy: ", test_accuracy)
72     endTime = time.time()
73     print('\n')
74     print(f"Total time: {endTime - startTime} seconds")
75 else:
76     distributed_logistic_regression(X_train, Y_train, lr, epochs)
77

```

فایل sh:

```

2 > الف > LogReg_a.sh
1  #!/bin/bash
2  #SBATCH --job-name=LogReg_a
3  #SBATCH --nodes=1
4  #SBATCH --ntasks-per-node=1
5  #SBATCH --partition=partition
6  #SBATCH --output=LogReg_a.out
7  echo 'jobs are starter ...'
8  srun --mpi=pmix_v4 python LogReg_a.py
9

```

نتیجه:

```
aqdam@raspberrypi-dm x + v
GNU nano 7.2 LogReg_a.out
jobs are starter ...
Accuracy: 0.959

Total time: 0.07452130317687988 seconds
```

براساس خروجی چاپ شده، دقت ۹۵٪ بر روی داده‌های تست در زمان ۰/۰۷ ثانیه بدست آمده است.

ب: این بار پیاده سازی بخش الف را همانگونه اجرا می‌کنیم با این تفاوت که باید روی یک نود و دو هسته باشد.

```
2 > ب > LogReg_b.py > ...
1 from mpi4py import MPI
2 import numpy as np
3 import time
4
5 def train_test_split_manual(X, Y, test_size=0.2):
6     data_size = X.shape[0]
7     test_size = int(data_size * test_size)
8     indices = np.random.permutation(data_size)
9     i_test = indices[:test_size]
10    i_train = indices[test_size:]
11    X_train, X_test = X[i_train], X[i_test]
12    Y_train, Y_test = Y[i_train], Y[i_test]
13    return X_train, X_test, Y_train, Y_test
14
15 def compute_gradient(X, Y, weights):
16     predicted = 1 / (1 + np.exp(-np.dot(X, weights)))
17     errors = Y - predicted
18     gradient = -np.dot(X.T, errors) / len(Y)
19     return gradient
20
21 def update_weights(weights, gradient, Lr):
22     weights -= Lr * gradient
23     return weights
24
25 def evaluate_model(X, Y, weights):
26     predicted = 1 / (1 + np.exp(-np.dot(X, weights)))
27     output = (predicted >= 0.5).astype(int)
28     accuracy = np.mean(output == Y)
29     return accuracy
30
```

2 > ب > LogReg_b.py > ...

```
30
31 def distributed_logistic_regression(X, Y, Lr, epochs):
32     comm = MPI.COMM_WORLD
33     rank = comm.Get_rank()
34     size = comm.Get_size()
35
36     total_features = X.shape[1]
37     weights = np.zeros(total_features)
38
39     data_size = X.shape[0]
40     sub_size = data_size // size
41
42     local_X = X[rank * sub_size:(rank + 1) * sub_size]
43     local_Y = Y[rank * sub_size:(rank + 1) * sub_size]
44
45     for epoch in range(epochs):
46         sub_gradient = compute_gradient(local_X, local_Y, weights)
47         total_gradient = np.zeros_like(sub_gradient)
48         comm.Allreduce(sub_gradient, total_gradient, op=MPI.SUM)
49         weights = update_weights(weights, total_gradient / size, Lr)
50
51     return weights
52
53 lr = 0.01
54 epochs = 100
55
56 startTime = time.time()
57
58 data = np.load('/home/dmls/aqdam/tmrine1/2/b/data.npy')
59 labels = np.load('/home/dmls/aqdam/tmrine1/2/b/labels.npy')
```

```

2 > ب > LogReg_b.py > ...
61
62 X_train, X_test, Y_train, Y_test = train_test_split_manual(data, labels)
63
64 comm = MPI.COMM_WORLD
65 rank = comm.Get_rank()
66
67 if rank == 0:
68     final_weights = distributed_logistic_regression(X_train, Y_train, lr, epochs)
69     test_accuracy = evaluate_model(X_test, Y_test, final_weights)
70     print("Accuracy: ", test_accuracy)
71     endTime = time.time()
72     print('\n')
73     print(f"Total time: {endTime - startTime} seconds")
74 else:
75     distributed_logistic_regression(X_train, Y_train, lr, epochs)
76

```

فایل ::sh

```

2 > ب > LogReg_b.sh
1  #!/bin/bash
2  #SBATCH --job-name=LogReg_b
3  #SBATCH --nodes=1
4  #SBATCH --ntasks-per-node=2
5  #SBATCH --partition=partition
6  #SBATCH --output=LogReg_b.out
7  echo 'jobs are starter ...'
8  srun --mpi=pmix_v4 python LogReg_b.py
9

```

نتیجه:

```
aqdam@raspberrypi-dm × + ∨
GNU nano 7.2 LogReg_b.out
jobs are starter ...
Accuracy: 0.956

Total time: 0.23209834098815918 seconds
```

که مشاهده می شود زمان اجرا نسبت به یک نود و یک هسته افزایش می یابد.

ج:

این بار کد را برای دو نود و دو هسته اجرا می کنیم.

2 > > LogReg_c.py > ...

```
1  from mpi4py import MPI
2  import numpy as np
3  import time
4
5  def train_test_split_manual(X, Y, test_size=0.2):
6      data_size = X.shape[0]
7      test_size = int(data_size * test_size)
8      indices = np.random.permutation(data_size)
9      i_test = indices[:test_size]
10     i_train = indices[test_size:]
11     X_train, X_test = X[i_train], X[i_test]
12     Y_train, Y_test = Y[i_train], Y[i_test]
13     return X_train, X_test, Y_train, Y_test
14
15  def compute_gradient(X, Y, weights):
16      predicted = 1 / (1 + np.exp(-np.dot(X, weights)))
17      errors = Y - predicted
18      gradient = -np.dot(X.T, errors) / len(Y)
19      return gradient
20
21  def update_weights(weights, gradient, lr):
22      weights -= lr * gradient
23      return weights
24
25  def evaluate_model(X, Y, weights):
26      predicted = 1 / (1 + np.exp(-np.dot(X, weights)))
27      output = (predicted >= 0.5).astype(int)
28      accuracy = np.mean(output == Y)
29      return accuracy
30
```

2 > > LogReg_c.py > ...

```
30
31 def distributed_logistic_regression(X, Y, Lr, epochs):
32     comm = MPI.COMM_WORLD
33     rank = comm.Get_rank()
34     size = comm.Get_size()
35
36     total_features = X.shape[1]
37     weights = np.zeros(total_features)
38
39     data_size = X.shape[0]
40     sub_size = data_size // size
41
42     local_X = X[rank * sub_size:(rank + 1) * sub_size]
43     local_Y = Y[rank * sub_size:(rank + 1) * sub_size]
44
45     for epoch in range(epochs):
46         sub_gradient = compute_gradient(local_X, local_Y, weights)
47         total_gradient = np.zeros_like(sub_gradient)
48         comm.Allreduce(sub_gradient, total_gradient, op=MPI.SUM)
49         weights = update_weights(weights, total_gradient / size, Lr)
50
51     return weights
52
53 lr = 0.01
54 epochs = 100
55
56 startTime = time.time()
57
58 data = np.load('/home/dmls/aqdam/tmrine1/2/c/data.npy')
59 labels = np.load('/home/dmls/aqdam/tmrine1/2/c/labels.npy')
```

```

2 > 2 > LogReg_c.py > ...
60
61 X_train, X_test, Y_train, Y_test = train_test_split_manual(data, labels)
62
63 comm = MPI.COMM_WORLD
64 rank = comm.Get_rank()
65
66 if rank == 0:
67     final_weights = distributed_logistic_regression(X_train, Y_train, lr, epochs)
68     test_accuracy = evaluate_model(X_test, Y_test, final_weights)
69     print("Accuracy: ", test_accuracy)
70     endTime = time.time()
71     print('\n')
72     print(f"Total time: {endTime - startTime} seconds")
73 else:
74     distributed_logistic_regression(X_train, Y_train, lr, epochs)
75

```

فایل .sh

```

2 > 2 > LogReg_c.sh
1  #!/bin/bash
2  #SBATCH --job-name=LogReg_c
3  #SBATCH --nodes=2
4  #SBATCH --ntasks-per-node=2
5  #SBATCH --partition=partition
6  #SBATCH --output=LogReg_c.out
7  echo 'jobs are starter ...'
8  srun --mpi=pmix_v4 python LogReg_c.py
9

```

نتیجه:

```

aqdam@raspberrypi-dm x + v
GNU nano 7.2 LogReg_c.out
jobs are starter ...
Accuracy: 0.955

Total time: 18.31640076637268 seconds

```

برخلاف تصور در این بخش زمان نسبت به مورد ب و الف شدیداً افزایش می‌یابد.

به نظر می‌رسد بدلیل استفاده از Allreduce در MPI برای جمع‌آوری اطلاعات از همه پردازش‌ها باعث افزایش تاخیر ارتباطی شده است و به همین خاطر زمان اجرا در افزایش هسته‌ها به ترتیب افزایش پیدا کرده است.